JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Map.Entry<K,V>

Type Parameters:

`K` - the type of the key

`V` - the type of the value

All Known Implementing Classes:

`AbstractMap.SimpleEntry`, `AbstractMap.SimpleImmutableEntry`

Enclosing interface:

`Map<K,V>`

`public static interface` **`Map.Entry<K,V>`**

A map entry (key-value pair). The Entry may be unmodifiable, or the value may be modifiable if the optional `setValue` method is implemented. The Entry may be independent of any map, or it may represent an entry of the entry-set view of a map.

An Entry maintains a connection to its underlying map if the Entry was obtained by iterating the `Map.entrySet()` view of a map, either explicitly by using an `Iterator` or implicitly via the enhanced `for` statement. This connection to the backing map is valid *only* during iteration of the entry-set view. During the iteration, if supported by the backing map, a change to an Entry's value via the `setValue` method will be visible in the backing map. The behavior of such an Entry is undefined outside of iteration of the map's entry-set view. It is also undefined if the backing map has been modified after the Entry was returned by the iterator, except through the `setValue` method. In addition, a change to the value of a mapping in the backing map might or might not be visible in the corresponding Entry of the entry-set view.

An Entry may also be obtained from a map's entry-set view by other means, for example, using the `parallelStream`, `stream`, `spliterator` methods, any of the `toArray` overloads, or by copying the entry-set view into another collection. It is unspecified whether the obtained Entry instances are connected to the underlying map, whether changes to such an Entry will affect the underlying the map and vice-versa, and whether such an Entry supports the optional `setValue` method.

In addition, an Entry may be obtained directly from a map, for example via calls to methods directly on the `NavigableMap` interface. An entry thus obtained is generally not connected to the map and is an unmodifiable snapshot of the mapping as of the time of the call. Such an Entry also does not generally support the `setValue` method.

An Entry obtained by direct construction of the `AbstractMap.SimpleEntry` or `AbstractMap.SimpleImmutableEntry` classes or from a call to the `Map.entry` or `Map.Entry.copyOf` methods is not connected to any map.

API Note:

The exact behavior of Entry instances obtained from a map's entry-set view other than via iteration varies across different map implementations; some are connected to the backing map, and some are not. To guarantee that an Entry is disconnected from its backing map, use the `copyOf` method. For example, the following creates a snapshot of a map's entries that is guaranteed not to change even if the original map is modified:

```
var entries = map.entrySet().stream().map(Map.Entry::copyOf).toList()
```

Since:

1.2

See Also:

`Map.entrySet()`

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
`static <K extends Comparable<? super K>, V> Comparator<Map.Entry<K,V>>`	**`comparingByKey`**`()`	Returns a comparator that compares `Map.Entry` in natural order on key.
`static <K, V> Comparator<Map.Entry<K,V>>`	**`comparingByKey`**`(Comparator<? super K> cmp)`	Returns a comparator that compares `Map.Entry` by key using the given `Comparator`.
`static <K, V extends Comparable<? super V>> Comparator<Map.Entry<K,V>>`	**`comparingByValue`**`()`	Returns a comparator that compares `Map.Entry` in natural order on value.
		Returns a comparator that compares `Map.Entry` by value

static <K, V> Comparator <Map.Entry<K,V>>	**comparingByValue** (Comparator<? super V> cmp)	using the given Comparator.
static <K, V> Map.Entry<K,V>	**copyOf**(Map.Entry<? extends K, ? extends V> e)	Returns a copy of the given Map.Entry.
boolean	**equals**(Object o)	Compares the specified object with this entry for equality.
K	**getKey**()	Returns the key corresponding to this entry.
V	**getValue**()	Returns the value corresponding to this entry.
int	**hashCode**()	Returns the hash code value for this map entry.
V	**setValue**(V value)	Replaces the value corresponding to this entry with the specified value (optional operation).

Method Details

getKey

K getKey()

Returns the key corresponding to this entry.

Returns:

the key corresponding to this entry

Throws:

IllegalStateException - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.

getValue

`V getValue()`

Returns the value corresponding to this entry. If the mapping has been removed from the backing map (by the iterator's `remove` operation), the results of this call are undefined.

Returns:

the value corresponding to this entry

Throws:

`IllegalStateException` - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.

setValue

`V setValue(V value)`

Replaces the value corresponding to this entry with the specified value (optional operation). (Writes through to the map.) The behavior of this call is undefined if the mapping has already been removed from the map (by the iterator's `remove` operation).

Parameters:

`value` - new value to be stored in this entry

Returns:

old value corresponding to the entry

Throws:

`UnsupportedOperationException` - if the `put` operation is not supported by the backing map

`ClassCastException` - if the class of the specified value prevents it from being stored in the backing map

`NullPointerException` - if the backing map does not permit null values, and the specified value is null

`IllegalArgumentException` - if some property of this value prevents it from being stored in the backing map

`IllegalStateException` - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.

equals

`boolean equals(Object o)`

Compares the specified object with this entry for equality. Returns `true` if the given object is also a map entry and the two entries represent the same mapping. More formally, two entries `e1` and `e2` represent the same mapping if

```
(e1.getKey()==null ?
 e2.getKey()==null : e1.getKey().equals(e2.getKey()))  &&
(e1.getValue()==null ?
 e2.getValue()==null : e1.getValue().equals(e2.getValue()))
```

This ensures that the `equals` method works properly across different implementations of the `Map.Entry` interface.

Overrides:

`equals` in class `Object`

Parameters:

`o` - object to be compared for equality with this map entry

Returns:

`true` if the specified object is equal to this map entry

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`int hashCode()`

Returns the hash code value for this map entry. The hash code of a map entry `e` is defined to be:

```
(e.getKey()==null   ? 0 : e.getKey().hashCode()) ^
(e.getValue()==null ? 0 : e.getValue().hashCode())
```

This ensures that `e1.equals(e2)` implies that `e1.hashCode()==e2.hashCode()` for any two Entries `e1` and `e2`, as required by the general contract of `Object.hashCode`.

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this map entry

See Also:

`Object.hashCode()`, `Object.equals(Object)`, `equals(Object)`

comparingByKey

```
static <K extends Comparable<? super K>, V>
Comparator<Map.Entry<K,V>> comparingByKey()
```

Returns a comparator that compares `Map.Entry` in natural order on key.

The returned comparator is serializable and throws `NullPointerException` when comparing an entry with a null key.

Type Parameters:

`K` - the `Comparable` type of then map keys

`V` - the type of the map values

Returns:

a comparator that compares `Map.Entry` in natural order on key.

Since:

1.8

See Also:

`Comparable`

comparingByValue

```
static <K, V extends Comparable<? super V>>
Comparator<Map.Entry<K,V>> comparingByValue()
```

Returns a comparator that compares `Map.Entry` in natural order on value.

The returned comparator is serializable and throws `NullPointerException` when comparing an entry with null values.

Type Parameters:

`K` - the type of the map keys

`V` - the `Comparable` type of the map values

Returns:

a comparator that compares `Map.Entry` in natural order on value.

Since:

1.8

See Also:

`Comparable`

comparingByKey

```
static <K,V> Comparator<Map.Entry<K,V>> comparingByKey
(Comparator<? super K> cmp)
```

Returns a comparator that compares `Map.Entry` by key using the given `Comparator`.

The returned comparator is serializable if the specified comparator is also serializable.

Type Parameters:

`K` - the type of the map keys

`V` - the type of the map values

Parameters:

`cmp` - the key `Comparator`

Returns:

a comparator that compares `Map.Entry` by the key.

Since:

1.8

comparingByValue

```
static <K,V> Comparator<Map.Entry<K,V>> comparingByValue
(Comparator<? super V> cmp)
```

Returns a comparator that compares `Map.Entry` by value using the given `Comparator`.

The returned comparator is serializable if the specified comparator is also serializable.

Type Parameters:

`K` - the type of the map keys

`V` - the type of the map values

Parameters:

`cmp` - the value `Comparator`

Returns:

a comparator that compares `Map.Entry` by the value.

Since:

1.8

copyOf

```
static <K,V> Map.Entry<K,V> copyOf
(Map.Entry<? extends K, ? extends V> e)
```

Returns a copy of the given `Map.Entry`. The returned instance is not associated with any map. The returned instance has the same characteristics as instances returned by the `Map::entry` method.

API Note:

An instance obtained from a map's entry-set view has a connection to that map. The `copyOf` method may be used to create a `Map.Entry` instance, containing the same key and value, that is independent of any map.

Implementation Note:

If the given entry was obtained from a call to `copyOf` or `Map::entry`, calling `copyOf` will generally not create another copy.

Type Parameters:

`K` - the type of the key

`V` - the type of the value

Parameters:

`e` - the entry to be copied

Returns:

a map entry equal to the given entry

Throws:

`NullPointerException` - if e is null or if either of its key or value is null

Since:

17

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.